

Analyse de Mixture of Experts pour la classification d'images

Auteur : Andrieu Ludovic

Abstract

Mon projet d'ARNA sur les mixtures of experts a consisté à d'abord étudier une première implémentation inspirée du papier [1] puis une seconde implémentation basée sur les idées dans le papier [2] et pour finir sur une analyse des performances des MoE avec des experts FFD simples comme dans le papier. Mon but a été de chercher les avantages et les limites de l'utilisation de MoE pour chacune des deux implémentations. J'ai réalisé mes expériences sur de la classification d'images avec Cifar-10 et 100 et j'ai mesuré de multiples métriques sur les performances et l'utilisation des experts par classes lors de l'entraînement. J'ai principalement obtenu comme résultats que sur Cifar l'utilisation de MoE ne permet pas améliorer l'accuracy du model sur le test mais que cela peut impacter l'overfitting du model. Les MoE ont impacté très négativement les temps d'apprentissage comparé aux models de base. Le routing des MoE est aussi très importante, car le choix entre une routing soft ou hard (top k) vas avoir un fort impacte en mémoire a l'entraînement et lors de l'inférence, cependant dans mes test je n'ai pas réussi a implémenter un routing hard qui été plus rapide que le soft.

1 Introduction et contexte

1.1 Introduction aux MoE

les MoE fonctionne grâce a un premier réseaux de routage qui a partir des données en entrées du MoE détermine une pondération sur les sorties des experts. Pour les experts il y a deux façon de les utiliser soit avec une routage "soft" c'est a dire que l'on vas calculer chaque expert sur toutes les données en entrées du MoE puis calculer la somme pondéré des sorties des experts par les activations par experts données par le routage. Le routage "hard" fonctionne ici grâce a un top K, pour dans ce rapport j'ai systématiquement utilisé $k = 2$ comme souvent suggéré dans le papier [2], ce routage ne fait pas passer toutes les données par l'ensemble des experts mais on fait un premier tri des K experts avec les meilleurs activations de routage pour chaque donnée du batch, ainsi les experts n'auront a traiter que les données ou ils ont été choisi sur le top K ce qui permet de garder la quantité de calcul independent du nombre d'experts en théorie. Le but des MoE est de pouvoir avoir un nombre de paramètres tres élevé tout gardant un besoin en calcul faible comparé au nombre de paramètres, ce qui est notement tres utile sur les LLM qui ont besoin d'une grand nombre de parametres. On vas aussi chercher a ce que les experts des MoE se specialises et donc ne donnent pas tous les meme resultats.

1.2 Context d'entrainements

Les entrainement se sont fait sur Cifar-10 principalement et quelques entrainement sur Cifar-100 on été réaliser pour permettre de confirmer les tendances observées sur Cifar-10. j'ai d'abord testé sur MNIST et MNIST-Fashion mais les données semblaient trop simples et l'écart de performances entre les différents essais n'était pas significatifs. Cifar-100 été une meilleur option mais il été beaucoup plus long a entrainer, j'ai donc fait le choix de rester sur Cifar-10. Par la suite j'ai envisager d'entrainer mon model sur ImageNet-1K mais l'implémentation de mes datasets n'été pas fait pour un dataset aussi gros car j'avait fait le choix de charger toutes les données du dataset pour accélérer par la suite les entrainement et donc faut d'avoir suffisamment de RAM, il m'étais pas possible de charger les nombreux gigas du split de test seulement. Dans un premier temps les entrainement ont été fait sans augmentation de données puis certain avec pour permettre de comparer.

2 Première approche : MoE CNN avec gating soft

2.1 Description du modèle

Cette première approche est donc inspirée du papier [1]. J'ai choisis de partir sur une architecture CNN classique sans utiliser de connexions résiduelles pour garder les résultats plus simple. Le model utilise un CNN pour le gating puis un CNN par expert. Pour les CNN il y a 3 configurations (small, medium, large) avec différent nombre de couches et paramètres. Cette approche n'as pas pour but d'être efficace en calcul mais d'être une introduction basique aux MoE avec une idée différente de ce que j'allais implémenter dans la deuxième partie. Ce MoE est donc un routage soft de tout un CNN ce qui le rends particulièrement coûteux lors que l'on augment le nombre d'experts, le coût vas être linéaire au nombre d'experts. Pour calculer la sortie du model, j'ai choisis de d'abord calculer les probabilités de chaque expert puis d'appliquer la somme pondérée a ces probabilités pour éviter les déséquilibres qui seraient engendrés si on utilisait les logits directement pour la somme pondérée.

2.2 Adaptation de la fonction de perte à la classification

Le papier [1] dont est inspiré cette approche est un problème de régression, or ici je suis sur de la classification d'images donc j'ai essayé de re-adapter la loss (1) initialement dans le papier pour qu'elle s'applique ici correctement. j'ai remarqué que le fin de l'équation été le calcul de la MSE, couramment utilisée en régression et je l'ai adapté en remplaçant cette partie par de la Cross Entropy, qui est adaptée pour la classification, ce qui donne la loss que j'ai premièrement utilisée (2). J'ai par la suite, a cause de problèmes d'équilibrages de l'utilisation des experts ajouté un terme de load balancing très simple (3). Puis en essayant de chercher a ameliorer la formule de la loss que j'avais adaptée, je me suis rendu compte que cette nouvelle formule était en fait grandement simplifiable comme montre dans l'équation (4) qui ressemble pratiquement juste au fait d'appliquer la Cross Entropy sur les probabilités du MoE au lieux de l'appliquer a chaque expert independement.

$$Loss = -\log(\sum_{i=1}^K g_i * \exp(-0.5 * ||y - o_i||^2)) \quad (1)$$

$$MSE(\hat{y}, y) = ||y - \hat{y}||^2 \quad CE_i(y) = -\log(p_{i,y})$$

$$Loss = -\log(\sum_{i=1}^K g_i * \exp(-0.5 * CE_i(y))) \quad (2)$$

$$Loss + MSE(\frac{1}{nbExperts}, meanGatePerExperts) \quad (3)$$

$$Loss = -\log(\sum_{i=1}^K g_i * \exp(\log[\sqrt{p_{i,y}}]))$$

$$\rightarrow Loss = -\log(\sum_{i=1}^K g_i * \sqrt{p_{i,y}}) \quad (4)$$

2.3 Analyse du routing et métriques

Pour pouvoir analyser le routing des experts il été important d'avoir une metrique qui me permeterais de mesurer certaines grandeures interessantes. Lors de chaque étapes de l'entrainement je sauvgarde le routing des experts de facon a garder dans l'historique l'utilisation moyenne de chaque expert durant l'epoche ainsi que l'utilisation moyenne de chaque expert en fonction de la classe prédit et la classe cible. Mais cela reste des matices de valeurs inexploitablees comme des metrics. j'ai donc choisit d'utiliser les deux metrics suivantes: GVofM et GMofV.

GVofM correspond a la variance de l'utilisation moyenne par experts au global, une valeur élevée correspond donc au fait qui il y a une forte disparité dans l'utilisation des experts.

le GMofV qui correspond a la moyenne des variances des activations des experts par classes ciblées, donc une valeur faible vas indiquer que pour chaques classes, tout les experts sont uniformement utilisés et une valeur élevé vas indiquer que l'actiation des experts est corrélée avec la classe a prédire.

Il est important de noter que les metriques données dans les tables sont celles mesurés sur les données de test, pas celle d'entrainement; de plus il est preferable de comparer ces metriques entre des resultats qui ont un meme nombre d'expert car j'ai oublié de normaliser les valeurs en tenant compte du nombre d'experts.

pour illustrer ces metriques on peut utiliser les activations sur les valeurs d'un des entrainement que l'on retrouve dans la table 1: gating "small" avec 4 experts "medium" avec la CrossEntropyLoss car elle donne des resultats diversifiés (ci dessous). on peut en effet voir que les experts 1, 2 sont sous utilisés et l'expert 5 est sur utilisé; de plus on peut voir que les classes 1 et 10 sont principalement prédites par un meme expert a chaque fois. On vas donc avoir un GVofM moderé car l'equilibre reste correcte mais on vas avoir un GMofV assé élevé car les experts choisit sont fortement corrélé avec la classe finale prédite.

Figure 1: matrice d'activations

les lignes : expert — les colonnes : classe cible

0.00	0.00	0.01	0.01	0.22	0.00	0.01	0.00	0.00	0.00
0.01	0.00	0.22	0.01	0.01	0.01	0.01	0.00	0.00	0.00
0.58	0.01	0.27	0.06	0.16	0.05	0.14	0.11	0.05	0.02
0.20	0.01	0.25	0.13	0.10	0.15	0.05	0.06	0.54	0.01
0.19	0.97	0.15	0.29	0.41	0.26	0.54	0.78	0.06	0.96
0.02	0.01	0.11	0.50	0.10	0.53	0.25	0.05	0.36	0.01

moyennes d'activations par expert : (GVofM = 0.145)

[0.021 0.030 0.144 0.153 0.456 0.197]

Std. des activations par expert : (GMofV = 0.159)

[0.065 0.063 0.164 0.149 0.319 0.192]

2.4 Discussion des résultats

Les principaux résultats obtenus avec la première approche sont dans la table 1 ci dessous. On peut constater que les models sans MoE sont considérablement plus rapide, c'était en effet attendu due a l'architecture choisit. Pour ce qui est de l'accuracy des models on vois que il s'agit systématiquement du model qui utilise la loss que j'ai adapté qui donne des

version	Train Acc.	Test Acc.	Δ Test Acc.	Time	GVofM	GMofV	# used*
small gating + 6 small experts, 223'458 parameters, 30 epochs							
base** small (32'362 p)	80.3%	74.8%	-0.7%	52 sec	–	–	–
loss adaptée	80.7%	75.5%	best	260 sec	0.373	0.000	1
loss adaptée + balance	89.8%	71.2%	-4.3%	265 sec	0.047	0.146	6
loss original	81.4%	75.0%	-0.5%	260 sec	0.332	0.052	2
cross entropy	86.8%	73.3%	-2.2%	278 sec	0.145	0.159	4 ~ 6
small gating + 12 small experts, 422'244 parameters, 30 epochs							
base** small (32'362 p)	80.3%	74.8%	-0.4%	52 sec	–	–	–
loss adaptée	80.9%	75.2%	best	459 sec	0.276	0.000	1
loss adaptée + balance	88.8%	72.2%	-3.0%	484 sec	0.118	0.070	5
loss original	82.4%	73.9%	-1.3%	459 sec	0.215	0.028	2
cross entropy	88.0%	74.8%	-0.4%	482 sec	0.127	0.078	4 ~ 5
small gating + 4 medium experts, 1'799'244 parameters, 10 epochs							
base** medium (442'874 p)	90.0%	78.7%	-1.2%	35 sec	–	–	–
loss adaptée	91.1%	79.9%	best	117 sec	0.433	0.000	1
loss adaptée + balance	90.2%	77.6%	-2.3%	123 sec	0.053	0.119	4
loss original	83.2%	74.3%	-5.6%	123 sec	0.274	0.205	2
cross entropy	86.3%	78.2%	-1.7%	128 sec	0.344	0.111	2 ~ 4

Table 1: Comparaison des différentes variantes de fonctions de perte pour la premiere aproche, en termes de performances, temps d'entraînement et métriques de routage.

used*: cela correspond au nombre d'experts reelement utilisés

base** "taille" (# parametres): sont les models de base, sans MoE, de meme taille que 1 expert

meilleurs résultats, qui sont donc meilleur que le model sans experts. Toutefois ces résultats sont a nuancer car le gain en résultats n'est pas suffisant pour justifier l'écart de temps d'entraînement, puis il faut aussi noter que cette version de la loss a pour effet de toujours converger vers l'utilisation d'un seul expert et ce des les premières epoques.

Il s'agit d'une conséquence logique de l'utilisation de cette loss adaptée car lors que l'on calcul le gradient de la loss pour l'activation de l'expert j on obtiens l'équation (5) où g_j est l'activation de l'expert et les p_j sont les probabilité prédite de la classe cible pour l'expert j . on peut donc voir que naturellement le model de routage vas router vers l'expert qui a le mieux prédit cette classe. En regardant l'équation (6) on vois que le gradient de la classe cible pour l'expert j est proportionnel a son activation g_j ce qui veux dire que si le model de routage ne vas pas tendre a équilibrer l'utilisation des experts mais favoriser uniquement l'expert qui donne les meilleurs résultats des le début puis abandonner les autres.

$$\frac{\partial Loss}{\partial g_j} = \frac{-\sqrt{p_j}}{\sum_{i=1}^K g_i * \sqrt{p_i}} \quad (5)$$

$$\frac{\partial Loss}{\partial p_j} = \frac{-0.5 * g_j}{\sqrt{p_j} * \sum_{i=1}^K g_i * \sqrt{p_i}} \quad (6)$$

Pour régler le problème d'équilibrage j'ai utilisé un terme supplémentaire (3) dans la loss. cela a eut pour effets d'utiliser plusieurs experts. Toutefois l'accuracy des models est grandement réduite avec cette ajout a la loss, le fait d'ajouter des experts dans la balance n'as pas amélioré les résultats. On peut aussi noter que lorsque l'on a beaucoup d'experts cela ne vas pas forcément tous les utiliser comme le montre le cas avec 12 experts où seulement 5 sont utilisés en pratique. Dans le papier [1] les auteurs affirment que leur loss permet un équilibrage automatique du routage des experts, j'ai donc implémenté aussi leur loss. Meme si il elle est initialement faite pour de la regression on peut quand meme l'utiliser ici. Les performances obtenues sont tres nuancées car l'accuracy des models obtenus est peu stable et en baisse par raport au models sans MoE, on peut toutefois noter que elle a systematiquement fait utiliser 2 experts.

Par la suite, en simplifiant la loss adaptée et donc en utilisant la Cross Entropy loss a la place, on obtient de bon resultats. En effet on peut directement constater que sans utiliser de load balancing supplémentaire, on utilise plusieurs experts et presque autant que avec la loss adaptée (2) + load balancing (3). Pour l'accuracy obtenue on peu toutefois noter que elle est en baisse par rapport aux autres méthodes mais cela reste acceptable et meilleur que avec la loss utilisant l'équilibrage. On peut aussi constater que plus les models utilisent d'expert et plus ils favorisent l'overfitting avec cette aproche.

3 Deuxième approche : MoE FFD avec routing hard

3.1 Description du MoE et du routing hard

Comme abordé précédemment le principale problème de la première approche est son manque total d'efficacité, car on y fait un routage soft avec des models compets pout le routage et chaque experts. Cela s'avère être très coûteux et mal optimisé les différents expert ne partages rien en commun. Dans le deuxième papier [2] l'approche est de garder un seul model CNN dans lequel le MoE est un layer qui s'inscrit dans la continuité du model. Ce layer fonctionne de la même façon avec un routeur puis des experts indépendants les uns des autres. Dans mon implementation j'ai reproduit le même layer que dans le papier, c'est à dire un gating avec un Linear simple puis un deuxième qui sert de noise qui va s'amenuiser au cours du temps; les experts sont des FFD avec une expansion x4 et un ReLu au milieu. Là où mon implementation se différencie c'est lors que elle est utilisée entre les couches convolutionnelles car on a des entrées sur plusieurs dimensions, j'ai fait le choix le plus simple qui est d'aplatir toute l'image sur le batch et de traiter les pixels indépendamment. l'efficacité de system sera testé plus en détails dans la section 4. Le routage est donc ici un routage hard top K comme dans le papier. En théorie il permet de grandement limiter le nombre de calculs et la consommation en mémoire, notamment de beaucoup moins dépendre du nombre d'experts ce qui permet d'augmenter grandement le nombre de paramètres en gardant un MoE très rapide et capable de tenir sur moins de VRAM. J'ai aussi repris la loss dans le papier, ils utilisaient déjà la Cross Entropy car ils étaient sur une LLM ainsi que un terme d'équilibrage bien plus avancé que celui que j'ai utilisé précédemment. L'équilibrage s'est révélé essentiel pour éviter au model de n'utiliser que un seul des experts.

3.2 Emplacement du MoE dans le réseau

Ce que j'ai principalement cherché à analyser dans cette section est l'impact de l'emplacement du layer de MoE dans le model. En effet j'ai pu constater que cela avait un impact important à plusieurs niveaux et les résultats obtenus seront expliqués dans la sous-section suivante 3.3. Étant donné que mon implémentation permet de l'intégrer entre les couches convolutionnelles je n'étais pas limité à l'utiliser à la fin du model après le flatten. J'ai donc utilisé plusieurs configurations qui sont les suivantes:

- **Versión 1:** Le MoE est placé après le flatten, et sa sortie est donc les logits des classes prédites.
- **Versión 2:** Le MoE est placé juste avant le flatten il est donc appliqué sur les derniers pixels du CNN.
- **Versión 3:** Le MoE est placé juste avant le dernier bloc de convolution, ses résultats par pixels seront donc utilisés dans une convolution.
- **Versión 4:** Le MoE est placé un bloc de convolution plus tôt que la V3, ce qui le place au milieu du CNN.
- **Versión 5:** il y a 2 MoE, placé respectivement comme la V3 et V4 pour évaluer le fait d'avoir 2 MoE.

3.3 Résultats expérimentaux

Je vais maintenant parler des résultats obtenus avec cette méthode, pour cela je vais utiliser les résultats présents dans la table 2 qui correspond aux résultats sur Cifar-10 et la table 3 pour les résultats sur Cifar-100.

Dans un premier temps je vais me concentrer sur Cifar-10 pour parler de l'accuracy, le temps d'entraînement ainsi que le routage des experts. Pour le temps d'entraînement, contrairement à ce à quoi je m'attendais avec le top K qui permettrait de garder un temps d'entraînement faible il a continué d'être très élevé. Plus particulièrement on peut voir avec les différentes versions que lors que le MoE est placé plus tôt dans le model, le temps d'entraînement explose, c'est dû au fait que au fur et à mesure du CNN le nombre de pixels est réduit, ce qui veut dire que lorsque aplatit ces pixels cela fait beaucoup varier la batch size à traiter par les experts. Pour l'accuracy, on peut voir que comme pour la première approche les MoE ne permettent pas d'améliorer les performances du model, toutefois il est important de noter que contrairement à la première méthode, ici les MoE n'introduisent pas significativement plus d'overfitting, voir certaines versions ont tendance à légèrement le réduire. Nous verrons toutefois que sur Cifar-100 l'impact sur l'overfitting sera plus marqué.

Lors d'un test fait en augmentant le nombre d'experts de 6 à 12 puis 24, cela n'a pas eu d'impact sur l'accuracy, le temps d'entraînement a augmenté ce qui sera expliqué dans la sous-section 4.1, de plus on voit que l'utilisation des experts reste très élevée ce qui montre à quel point la méthode d'équilibrage utilisée dans ce papier est capable de gérer la mise à l'échelle. Maintenant nous allons regarder le GMoFV pour analyser son évolution pour chacune des versions. Tout d'abord il est important de noter que étant donné que l'on compare des models avec le même nombre d'experts, il est pertinent de comparer cette mesure entre eux. Ce que l'on peut dire est que le GMoFV diminue considérablement, en passant de 0.2 à 0.02 soit 10x moins, au plus on place le MoE loin de la fin du model. Comment peut-on expliquer cette diminution ? Étant donné que cette mesure à quel point le choix de l'expert est corrélé avec la classe cible. Or le rôle du routage est de séparer les données par experts linéairement et ne tient à aucun moment compte de la classe cible. La diminution du GMoFV est donc un comportement émergent de l'information contenue dans les données à chaque étape du model. On en déduit que plus on est proche de la sortie du model et plus la façon la plus naturelle de séparer les données linéairement s'apparente à prédire la classe de l'image.

Les résultats sur Cifar-100 sont très intéressants notamment quand on s'intéresse à l'overfitting. On peut voir un overfitting

Config	#experts	top K	Param.	Train Acc.	Test Acc.	Δ Test Acc.	Time	GVofM	GMofV	% Used
sans data augmentations, medium model, 15 epoques (baseline: V1 medium + 1 expert)										
V1	1	-	497'916	92.47%	79.27%	0.00	56.0 sec	-	-	100%
V1	6	2	855'096	92.25%	80.27%	+1.00	79.3 sec	0.035	0.221	100%
V2	6	2	2'113'286	96.12%	78.12%	-1.15	84.5 sec	0.037	0.118	100%
V3	6	2	1'127'558	89.99%	79.04%	-0.23	122.4 sec	0.017	0.032	100%
V4	6	2	535'046	90.72%	79.39%	+0.12	341.8 sec	0.031	0.020	100%
V5	6	2	1'326'866	82.64%	76.56%	-2.71	351.8 sec	0.006	0.008	100%
<i>2nd layer of MoE</i>										
								0.036	0.046	100%
avec data augmentations, medium model, 25 epoques (baseline: V1 medium + 1 expert)										
V1	1	1	497'916	82.36%	78.74%	0.00	138.1 sec	-	-	100%
V1	6	2	855'096	81.49%	77.38%	-1.36	182.8 sec	0.041	0.199	100%
V2	6	2	2'113'286	84.26%	78.72%	-0.02	180.6 sec	0.014	0.129	100%
V3	6	2	1'127'558	80.10%	76.22%	-2.52	251.6 sec	0.029	0.035	100%
V4	6	2	535'046	81.24%	76.75%	-1.99	514.1 sec	0.018	0.020	100%
V5	6	2	1'326'866	76.35%	72.77%	-5.97	665.4 sec	0.013	0.014	100%
<i>2nd layer of MoE</i>										
								0.023	0.049	100%
V2	12	2	3'890'834	83.35%	77.31%	-1.43%	228.0 sec	0.019	0.074	100%
V2	24	2	7'445'930	83.02%	77.19%	-1.55%	319.4 sec	0.020	0.044	91.7%
avec data augmentations, large models, 25 epoques (baseline: V1 large + 1 expert)										
V1	1	1	2'395'916	86.94%	82.25%	0.00	226.9 sec	-	-	100%
V1	6	2	3'177'416	86.76%	81.94%	-0.31	256.7 sec	0.030	0.215	100%
V2	6	2	4'422'614	86.86%	81.17%	-1.08	267.8 sec	0.021	0.139	100%
V3	6	2	4'422'614	84.23%	79.58%	-2.67	380.9 sec	0.025	0.048	100%
V5	6	2	5'214'434	81.60%	77.12%	-5.13	873.9 sec	0.010	0.022	100%
<i>2nd layer of MoE</i>										
								0.029	0.057	100%

Table 2: Comparaison des différentes configurations MoE pour la seconde approche.
 Δ Test Acc. est calculé relativement au modèle baseline à 1 expert de même taille.

Config	#experts	top K	Param.	Train Acc.	Test Acc.	Δ Test Acc.	Time	GVofM	GMofV	% Used
sans data augmentations, 30 epoques (baseline: V1 medium + 1 expert)										
V1	1		544'086	56.72%	46.87%	0.00	161.9 sec	-	-	100%
V1	6	2	1'132'116	63.64%	42.18%	-4.69	190.0 sec	0.075	0.161	83.3%
V3	6	2	1'196'768	49.92%	42.04%	-4.83	296.3 sec	0.031	0.085	100%

Table 3: Résultats des configurations MoE sans data augmentation sur Cifar-100.

important déjà sur le model de reference, sur les models avec MoE on retrouve la chute d’accuracy sur le test mais les deux Versions ont la meme performance de test. Par contre la version V3 a considerablement moins d’overfing que les deux autres versions, là ou la V1 augmente l’overfitting.

4 Analyse des performance et efficacité des différents routage

cette section est dédiée a comparer les routage ”soft” contre le routage ”hard” top K que j’ai implementé a partir du deuxieme papier [2] et qui ont servi pour les resultats de la sous partie 3.3. En realité pour les resultats de cette sous partie j’ai utilisé un routage top K mais sans effectuer l’optimisation qui consiste ne donner a chaque experts que ses données pour des raison d’efficacité de mon implementation du routage ”hard” qui s’est averé sinificativement plus lent en pratique comme il sera expliqué dans la suite de cette partie.

4.1 Mise a l’échelle du nombre d’experts

Le routage ”soft” parait en theorie pire que le routage ”hard” avec top K en tout points en effets on effectue batchSize * nbExperts calculs alors que le routage hard effectue batchSize * K, sachant que K est fixé et souvent bien inferieur au nombre d’experts le temps de calcul et la consommation de memoire pourrait etre fix quand on augmente les experts.

En pratique, comme il on peut le voir dans la table 4 qui mesure le temps et la memoire utilisée par le MoE a partir du top K (car le top K ne change pas donc il est exclu des mesures), dans cette experiance les test sont fait sur le GPU et en ayant les gradients d’activés ici. Tout d’abord on peut largement voir que la methode avec le routage ”hard” est bien plus lente que le routage ”soft” mais par contre il a la particularité que avec les gardient activé (donc pendant l’apprentissage) il consomme considerablement moins de memoire, en fait la memoire augmente tres peu avec le nombre d’experts, ce qui permet dans la derniere ligne de pouvoir utiliser un MoE de 2.15B parametres sur moins de 12Go de VRAM en fp32 au lieu d’avoir besoin de 1.7To de VRAM, ce qui reviendrait un peu cher. Les chiffres de temps par element et par expert me

Batch size	#experts	#feat.	# param.	Time	Time2*	Peak mem.	Time ratio	Mem ratio
12544	6	64	198'528	3.713 ms	49 ns	64 Mo	4.35×slower	2.16×less
12544	24	64	794'112	11.261 ms	37 ns	62 Mo	3.62×slower	7.84×less
12544	64	64	2'117'632	34.244 ms	43 ns	71 Mo	3.30×slower	17.82×less
12544	128	64	4'235'264	70.829 ms	44 ns	86 Mo	2.36×slower	29.22×less
12544	256	64	8'470'528	118.720 ms	37 ns	115 Mo	1.30×slower	43.55×less
12544	512	64	16'941'056	257.920 ms	40 ns	174 Mo	1.18×slower	57.55×less
65536	6	256	3'153'408	15.938 ms	41 ns	1'062 Mo	11.57×slower	2.55×less
65536	24	256	12'613'632	23.315 ms	15 ns	1'061 Mo	4.81×slower	9.34×less
65536	64	256	33'636'352	41.253 ms	9.4 ns	1'136 Mo	–	~22×less**
65536	4096	256	2'152'726'528	2'965.0 ms	11 ns	11'714 Mo	–	~145×less**

Table 4: comparaison du coût en calcul et en mémoire du routage "hard" contre le routage "soft" en fonction du nombre d'experts.

Time2*: c'est le temps passé par element du batch et part expert, pour mesurer l'efficacité.

** : ces configurations la ont besoin de top de memoire et donc la consommation a été estimé en faisant une regression a partir de multiples autres valeurs (non présentes dans la table).

Mode	Batch size	#experts	#feat.	#param.	Time	Peak mem.	FLOPs
With gradients							
Hard routing	12544	256	128	33'784'320	124.89 ms	705 Mo	8.23 GFLOPs
Soft routing	12544	256	128	33'784'320	101.865 ms	10'478 Mo	845.10 GFLOPs
Soft routing V2	12544	256	128	33'784'320	111.419 ms	8'853 Mo	845.10 GFLOPs
Simple FFD	12544	–	128	33'784'320	29.922 ms	13'747 Mo	845.93 GFLOPs
Without gradients							
Hard routing	12544	256	128	33'784'320	110.103 ms	619 Mo	8.23 GFLOPs
Soft routing	12544	256	128	33'784'320	45.497 ms	3'876 Mo	845.10 GFLOPs
Soft routing V2	12544	256	128	33'784'320	49.728 ms	639 Mo	845.10 GFLOPs
Simple FFD	12544	–	128	33'784'320	27.489 ms	13'747 Mo	845.93 GFLOPs

Table 5: comparaison du coût en calcul et en mémoire du routage "hard" contre le routage "soft" a l'inference et durant l'entrainement.

font dire que le probleme de lenteur du routage hard vient probablement de mon implementation qui n'est pas adaptée aux GPU, car on peut constater que le temps par expert reste constant a partir d'un certain nombre d'experts, ce qui veut dire que le temps de calcule est proportionel aux experts ce ui ne devrait pas etre le cas avec un top K.

4.2 Comparaison entre l'inference et l'entrainement

La table 5 met en avant beaucoup d'autre chose car il y a des changements lorsque l'on desactive les gradients. Tout d'abord pour expliquer les differents Mode, "Simple FFD" sert a comparer les MoE avec 1 FFD similaire aux experts mais avec une expansion considerable pour qu'il soit de taille equivalent a tout le MoE. De plus il y a une V2 du routage "soft" car il s'agit d'une re-ecriture du soft routage que j'ai fait tardivement dans le projet elle n'est donc présente que sur cette table car etant donné que elle ne modifie pas les conclusion des autres resultats qui sont fait avec les gardients je n'ai pas refait toutes mes experiences avec. Cette table illustre effectivement que la lenteur du routage "hard" est surprenante, en effet comme on peu le voir avec les FLOPs de chaques methodes, il es censé etre beaucoup plus rapide avec près de 100 fois moins de puissance requise mais en pratique il est pourtant sinificativement plus lent malgré le grand nombre d'experts et surtout a l'inference. On peut noter que avec les gradients les routage "soft" et "hard" sont presque equivalent en temps de calcul mais que lors de l'inference ils sont netement plus rapide car le temps du "hard" diminue peu, ce qui renforce mon hypothèse d'une implementation mal optimisée. Pour la memoire consommée le "hard" en consomme tres peu dans tout les cas, par contre le soft (V2) en consomme beaucoup avec les gradients q'il faut conserver mais sans ses gradients il consomme a peine plus que le "hard"; le FFD simple quand a lui est beaucoup plus rapide a calculer mais consomme le plus de memoire.

4.3 Comparaison entre le GPU et CPU

Dans cette sous section on vas regarder les résultats obtenus sur CPU sur la table 6 car ils permettent de bien mètre en avant la capacité du routage "hard" a etre plus rapide que le routage "soft". On peut tout d'abord en effet constater que sur CPU les temps de calculs sont bien plus élevés que sur GPU ce qui est normal car le GPU fait beaucoup plus de choses en parallèle. Mais toute fois on peu remarquer les temps pour le routage "hard" sur CPU sont bien inférieurs

Mode	Batch size	#experts	#feat.	#param.	CPU	GPU	Memory*	FLOPs
Hard routing	12544	6	128	791'820	52.13 ms	7.62 ms	131 Mo	6.63 GFLOPs
Hard routing	12544	64	128	8'446'080	78 ms	38.32 ms	261 Mo	7.00 GFLOPs
Hard routing	12544	256	128	33'784'320	189 ms	149.60 ms	705 Mo	8.23 GFLOPs
Soft routing	12544	6	128	791'820	80.2 ms	3.47 ms	266 Mo	19.81 GFLOPs
Soft routing	12544	64	128	8'446'080	1'117 ms	24.03 ms	2'636 Mo	211.28 GFLOPs
Soft routing	12544	256	128	33'784'320	5'081 ms	89.10 ms	10'478 Mo	845.10 GFLOPs
Soft routing V2	12544	6	128	791'820	62.4 ms	3.47 ms	247 Mo	19.81 GFLOPs
Soft routing V2	12544	64	128	8'446'080	752 ms	26.92 ms	2'244 Mo	211.28 GFLOPs
Soft routing V2	12544	256	128	33'784'320	3'501 ms	119.60 ms	8'853 Mo	845.10 GFLOPs
Simple FFD	12544	~ 6	128	791'945	72.0 ms	1.17 ms	337 Mo	19.83 GFLOPs
Simple FFD	12544	~ 64	128	8'446'176	983 ms	7.88 ms	3'448 Mo	211.48 GFLOPs
Simple FFD	12544	~ 256	128	33'784'320	3'780 ms	27.60 ms	13'747 Mo	845.93 GFLOPs

Table 6: Comparaison CPU/GPU des coûts en temps, mémoire et FLOPs entre routage dur, routage doux et couche dense équivalente pour différents nombres d’experts.

Memory*: Il s’agit de la memoire GPU utilisée car je n’ai pas réussi a mesurer le memoire CPU aussi specifiquement.

a ceux des autres méthodes et qu’ils sont du même ordre de grandeur que sur GPU, ce qui continue a confirmer que il y a un probleme sur mon implementation car avec 256 experts les temps CPU/GPU sont quasi identiques car sur CPU le temps de calcul croit plus lentement que sur GPU. On peu aussi constater que les FLOPs des autres methodes augment tres rapidement comme le temps de calcul. La memoire utilisée par le "soft" V2 suit la meme croissance que la version de base ce qui n’invalide pas les resultats montrés par la table 4.

5 Discussion critique

Lorsque j’ai abordé pour la premiere fois le sujet et surtout a la vue des differents papier que j’ai lue, je m’attendait a ce que l’effet des premier des MoE soit d’améliorer l’accuracy du model au cout d’une léger augmentation avec la methode top K. Sauf que en faisant mes experiances je me suis rendu compte que cette amelioration tel que le montrait les papiers dépendait dans une premier temps d’un dataset propice a ce que le MoE puisse capitaliser dessus. En effet les dataset que j’ai utilisé, Cifar-10 et 100, sont beaucoup trop petits et ne continnent pas assé de classes a prédire tel que l’on pourrait le retrouver dans ImageNet-1K ou bien un dataset pour entrainer des LLM. Ces datasets ont des milier de classes a prédire et des milion voir milliard de données que le model doit condenser. Et c’est la que les MoE jouent un role, en effet sur ces dataset les données ne seront probablement pas autant séparées par les classes a prédire de part leur nombre et la diversité en sortie, contrairement a ce que j’ai pu constater dans la sous section 3.3 des résultats de la seconde approche. De plus, meme si dans mon cas avoir plus de parametre n’est pas ce qui pouvait améliorer le model car le dataset est le facteur limitant, pour les problemes de language où le fait d’avoir plus de parametres donne de meilleurs resultats, comme on le retrouve dans le papier [2] est un excelent cas pour utiliser les MoE car ils permettent d’augmenter le nombre de parametres sans augmenter autant la quantité de ressources necessaires. pour arriver a ces ameliorations il faut toutefois s’assurer que les experts soient utilisés et que il apportent une qualité supplémentaire chacun les un des autres, pour cela il aurait été interesant de chercher une methode pour aider le model a maximiser la similarité entre les reponses des experts.

6 Conclusion et perspectives

Ce projet m’a vraiment permis de me pencher sur les MoE. Je me suis notement confronté a un cas ou les MoE ne se sont pas montré capable d’apporter des ameliorations ce qui meme si cela m’as au début déplus, m’as forcé a explorer tout le reset. J’en suis venu a chercher une explication sur les raison des mauvaises performances que j’ai obtenue et a trouver des solutions pour regler les erreurs que j’avait fait dans mon adaptation de la premiere aproche. Dans la seconde aproche après avoir constaté que je n’obtiendrait pas de meilleurs resultats j’ai chercher a exploiter les MoE pour comprendre leur fonctionnements et les comportement émergents du routage en fonction de l’emplacement du MoE dans le model. J’aimerais poursuivre ce travail en entrainant mes models sur ImageNet-1K pour lequel j’ai manqué le temps de finir d’adapter mes dataset pour supporter un aussi gros jeux de donné. De plus il serait interesant de chercher a apliquer de MoE a un model de language pour essayer d’en améliorer les performances. Et pour finir je shouaiterais tout particulièrement finir par comprendre pourquoi mon routage "hard" s’est averé etre aussi lent, la où dans les papiers ils semblent rendre le model plus rapides.

References

- [1] Jacobs et al., Adaptive Mixtures of Local Experts, Neural Computation, 1991.
- [2] Shazeer et al., Outrageously Large Neural Networks : The Sparsely-Gated Mixture-of-Experts Layer, ICLR 2017.